Experiment 05: Controlling the Sensor Peripherals from the Cloud Platform (Thingspeak)

Roll Number: 21BIT191

Objective: IoT Control using Cloud Platform

Equipment and Components: Raspberry Pi – 4B, SD card/ USB Drive, Laptop, SD card reader (optional), jumper wires, LED, Servo Motor, ThingSpeak Cloud Platform Account

Theory: In this experiment, we explore the concept of the Internet of Things (IoT) by controlling physical hardware components such as a servo motor and an LED remotely using a Raspberry Pi. The Raspberry Pi is interfaced with these components, and the control commands are sent from a cloud environment, specifically using ThingSpeak, a popular IoT platform. ThingSpeak allows us to send data to and receive data from the Raspberry Pi over the internet, enabling remote control. In this case, the servo motor's position and the LED's on/off state are determined by the data fields in the ThingSpeak channel, which are updated in real-time. This setup demonstrates how IoT can be used to remotely monitor and control physical devices, bridging the gap between cloud computing and hardware interaction.

Interfacing Servo Motor with Raspberry Pi

To interface the Servo Motor with the Raspberry Pi, three connections are required:

1. **Signal**: Connected to the any of the GPIO Raspberry Pi. In our case its GPIO 21.
2. **Power**: Connected to 5V pin of the Raspberry Pi.
3. **GND**: Connected to the ground pin of the Raspberry Pi.

Python Script for Reading Data

This Python code consists of two parts that enable remote control of an LED and a servo motor connected to a Raspberry Pi using the ThingSpeak platform. The first script sets up GPIO pins on the Raspberry Pi, where one pin controls an LED and another controls a servo motor via PWM (Pulse Width Modulation). The code continuously retrieves data from a ThingSpeak channel using its API, reading the latest values for LED state and servo angle from the fields in the channel. Based on the data, the LED is turned on or off, and the servo motor is positioned to the specified angle. The program runs in a loop, periodically fetching updates from ThingSpeak, and it handles keyboard interrupts gracefully by stopping the PWM and cleaning up the GPIO pins.

The second part of the code demonstrates how to send data to the ThingSpeak channel. It constructs a URL with the `WRITE_API_KEY` and updates the channel's fields with new values for controlling the LED and servo motor. These values are sent to ThingSpeak using an HTTP GET request, allowing remote updates to the Raspberry Pi's hardware components.

Setting Up the ThingsSpeak Account:

Step 1: Create a ThingSpeak Account (Using an Existing MathWorks Account)

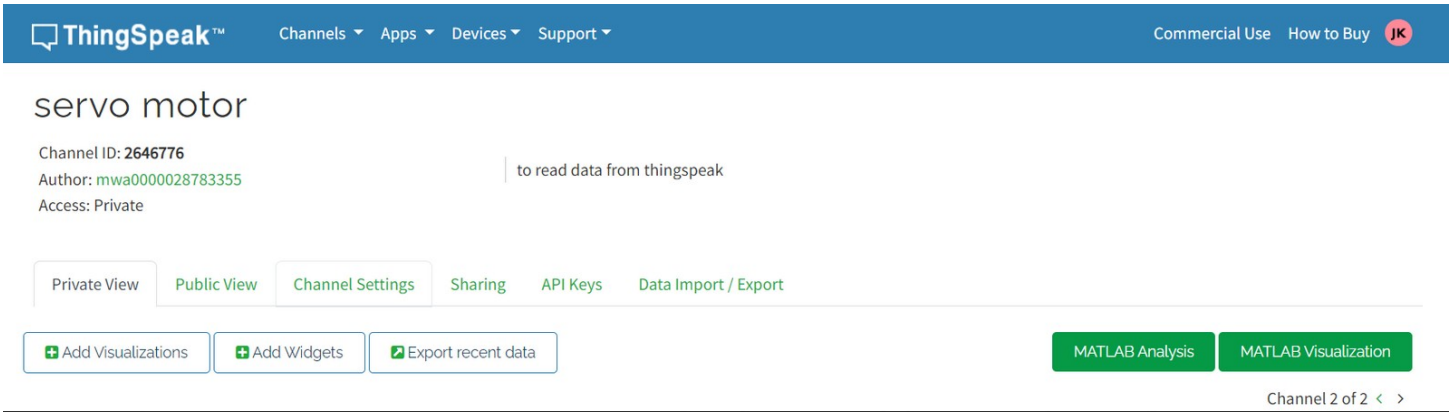
1. Visit the ThingSpeak Website:
    - Open your web browser and go to ThingSpeak's website.
2. Sign In with Your MathWorks Account:
    - Click on the "Sign In" button at the top right corner of the page.
    - You'll be redirected to the MathWorks login page. Enter your MathWorks credentials (email and password).
    - After successfully logging in, you'll be redirected back to ThingSpeak, and your ThingSpeak account will be automatically created.

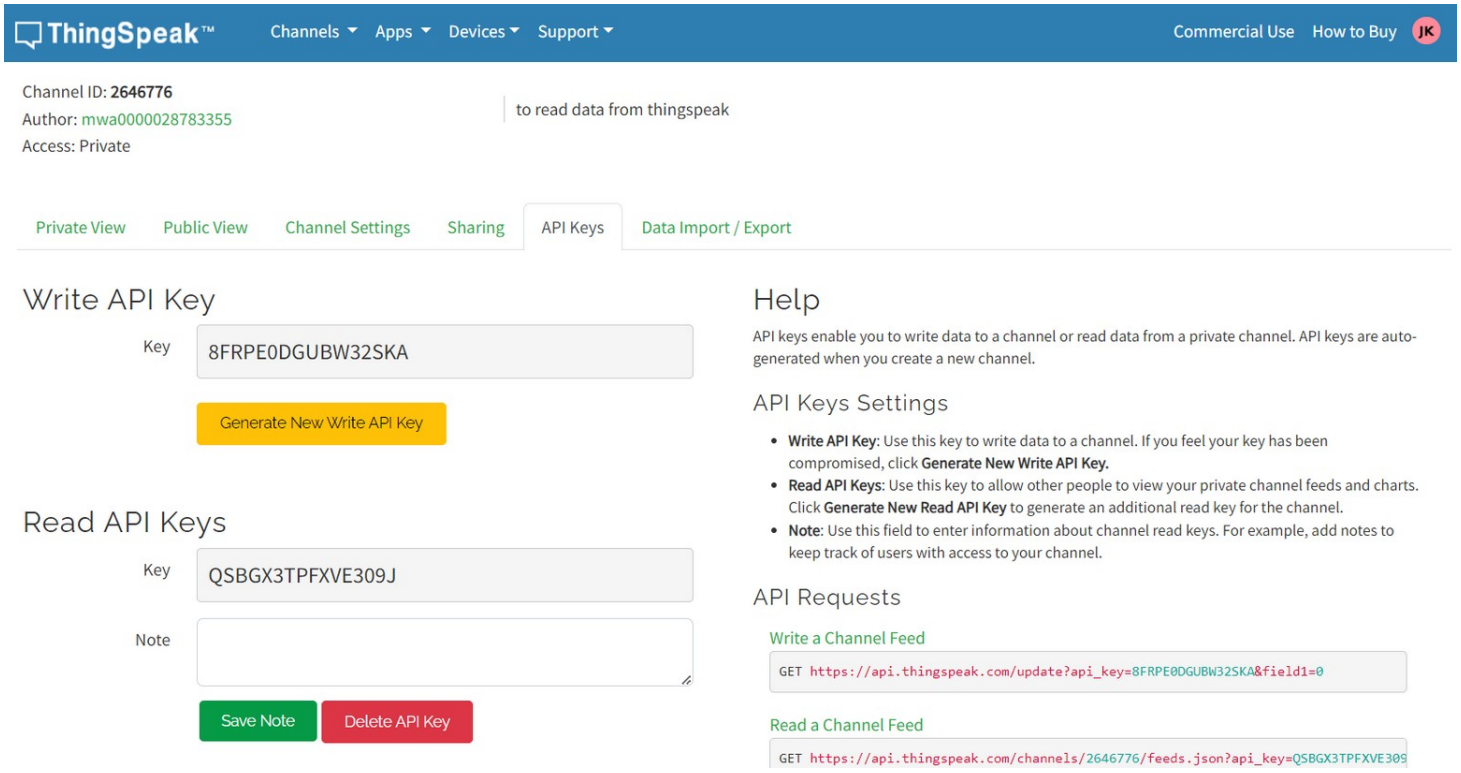
Step 2: Create a New Channel

1. Navigate to the "Channels" Tab:
    - Once logged in, look at the top navigation bar and click on "Channels".
2. Create a New Channel:
    - In the "Channels" tab, click on the "New Channel" button.
3. Fill in Channel Details:
    - Name: Enter a name for your channel.
    - Description: (Optional) Enter a description
    - Field 1: Enter the name "LED".
    - Field 2: Enter the name "Servo Motor".
    - Leave other fields as default unless you have specific needs.
4. Save the Channel:
    - Scroll down and click the "Save Channel" button.

Step 3: Get Your Write API Key

1. Go to the "API Keys" Tab:
    - After saving your new channel, you'll be redirected to the channel's page.
    - Navigate to the "API Keys" tab on this page.





2. Copy the Write API Key:
    - Under the "API Keys" tab, you'll see a section labeled "Write API Key".
    - Copy the Write API Key. This key will be used to send data to your ThingSpeak channel.

Procedure:

1. After setting up raspberry pi in experiment 2, now you can direct open PuTTY and repeat same steps as done previously. So, the Raspberry Pi OS desktop will appear.

2. Now, we need to update our Raspberry Pi. Run the following command to update and upgrade your Raspberry Pi to ensure that you have the latest software and libraries:

   sudo apt update && sudo apt upgrade

   If there are updates available, the system will prompt you to confirm. Press Y and Enter to proceed with the update. This process may take a few minutes.

3. We need to make an Virtual Environment for the Python. Navigate to your Desktop directory using the Terminal:
   cd ~/Desktop

   Create a new folder for your project. This folder will contain your virtual environment and the Python code:
   mkdir dht_test
   Move into the newly created directory:
   cd ~/Desktop/vmfolder

   Create a virtual environment within this directory named myenv (or any other name you prefer):
   python3 -m venv myenv

   Activate the virtual environment:
   source myenv/bin/activate

   After activation, your terminal prompt will change, indicating that you are now working within the virtual environment.

   4. Also we need to download the request library in the virtual environment so install it by form terminal of myenv using:

      pip install requests

5. Then Write the Python Script. In the same directory where you created the virtual environment (~/Desktop/vmfolder), create a new Python file, for example, servo.py.
   Open the file in a text editor and copy the following Python code into it:
   Save the file and close the editor.

6. Further, Run the Python Script. Ensure that the virtual environment is active. If it's not, reactivate it by running: source myenv/bin/activate

   Run the Python script using the following command: python servo.py. After the Successful interpretation of the Code it will prompt as the Data is successfully sent to ThingSpeak. After 15 seconds the data will be visible in the ThingSpeak Platform. Now to change the angle of the Servo Motor you can change the code in the speak.py file and run from the terminal. To turn off the LED you can set the LED as 0. Same should be reflected in the Hardware also.

   Servo.py

```python
import RPi.GPIO as GPIO
import requests
import time

# Setup GPIO pins
LED_PIN = 4
SERVO_PIN = 21
GPIO.setmode(GPIO.BCM)
GPIO.setup(LED_PIN, GPIO.OUT)
GPIO.setup(SERVO_PIN, GPIO.OUT)

# Servo PWM setup
pwm = GPIO.PWM(SERVO_PIN, 100)  # 50Hz frequency
pwm.start(0)

# ThingSpeak API details
CHANNEL_ID = '2646776'
WRITE_API_KEY = '8FRPE0DGUBW32SKA'
READ_API_KEY = 'QSBGX3TPFXVE309J'
READ_API_URL = f'https://api.thingspeak.com/channels/{CHANNEL_ID}/feeds.json?api_key={READ_API_KEY}&results=1'

def set_led(state):
    GPIO.output(LED_PIN, GPIO.HIGH if state == '1' else GPIO.LOW)

def set_servo(angle):
    duty = 2 + (angle / 18)
    GPIO.output(SERVO_PIN, True)
    pwm.ChangeDutyCycle(duty)
    time.sleep(0.5)
    GPIO.output(SERVO_PIN, False)
    pwm.ChangeDutyCycle(0)

try:
    while True:
        response = requests.get(READ_API_URL)
        data = response.json()
        led_state = data['feeds'][0]['field1']
        servo_angle = int(data['feeds'][0]['field2'])

        set_led(led_state)
        set_servo(servo_angle)

        time.sleep(15)  # Wait before the next update
except KeyboardInterrupt:
    print("Experiment stopped")
finally:
    pwm.stop()
    GPIO.cleanup()
```

speak.py

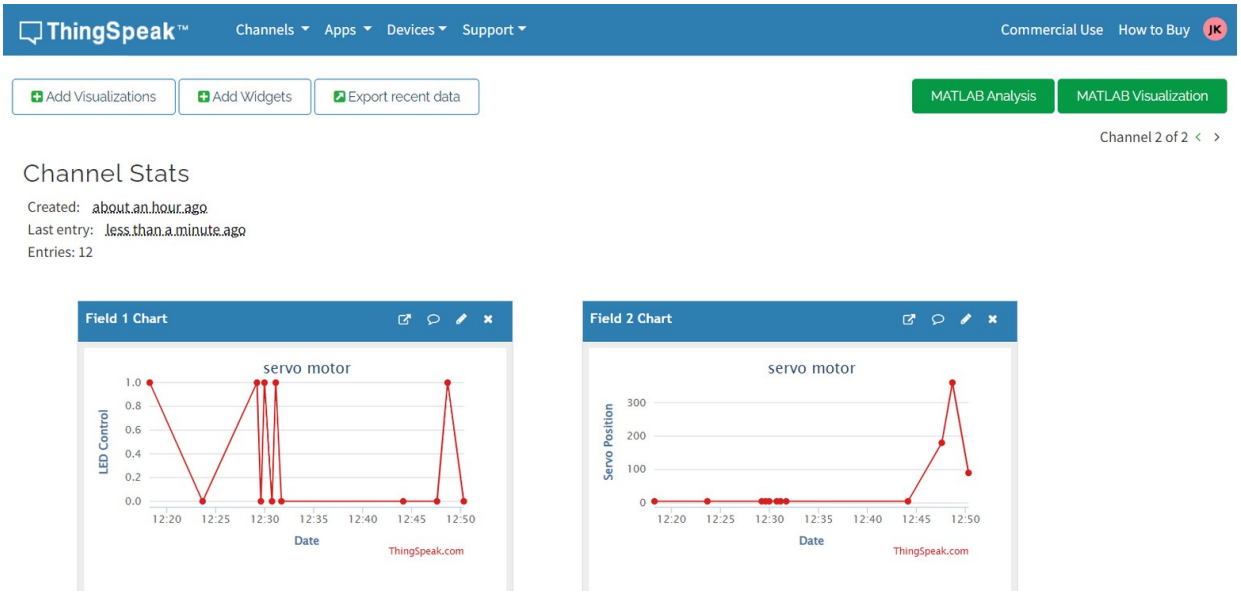
```python
import requests

write_api_key = '8FRPE0DGUBW32SKA'
field1_value = 0  # Example value for LED Control
field2_value = 90  # Example value for Servo Position

url = f"https://api.thingspeak.com/update?api_key={write_api_key}&field1={field1_value}&field2={field2_value}"
response = requests.get(url)
print(response.text)
```

Run the Command.

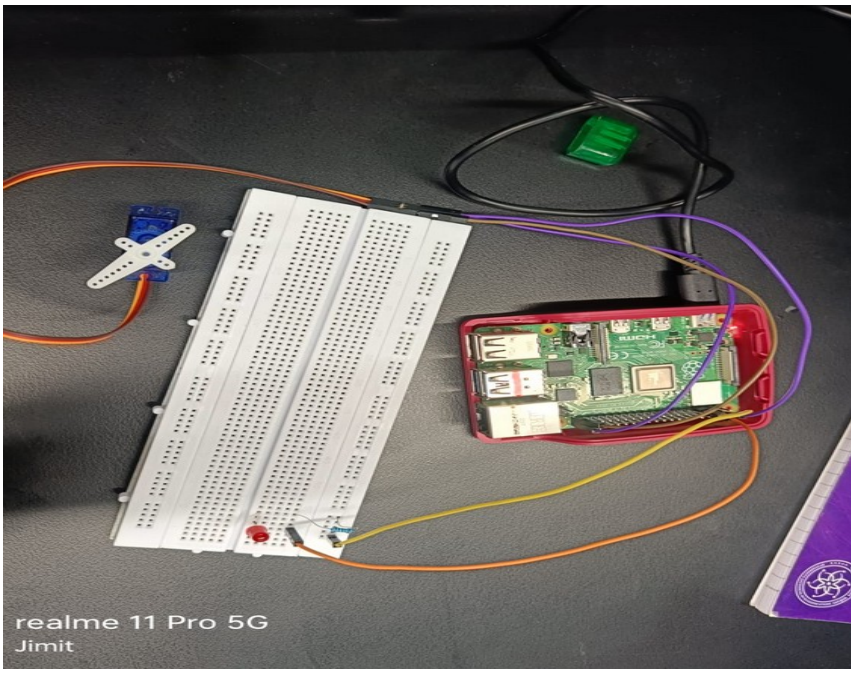
```
jjkk@raspberrypi: ~/vmfolder/myenv
File  Edit  Tabs  Help
jjkk@raspberrypi:~/vmfolder/myenv $ ls
bin  include  lib  lib64  onledcloud.py  pyvenv.cfg  servo.py  speak.py
jjkk@raspberrypi:~/vmfolder/myenv $ python speak.py
9
jjkk@raspberrypi:~/vmfolder/myenv $ python servo.py
/home/jjkk/vmfolder/myenv/servo.py:9: RuntimeWarning: This channel is already in
 use, continuing anyway.  Use GPIO.setwarnings(False) to disable warnings.
  GPIO.setup(LED_PIN, GPIO.OUT)
/home/jjkk/vmfolder/myenv/servo.py:10: RuntimeWarning: This channel is already i
n use, continuing anyway.  Use GPIO.setwarnings(False) to disable warnings.
  GPIO.setup(SERVO_PIN, GPIO.OUT)
n
```

```
(myenv) jjkk@raspberrypi:~/vmfolder/myenv $ python speak.py
1
(myenv) jjkk@raspberrypi:~/vmfolder/myenv $ python servo.py
```

Result:



Roll Number: 21BIT191



Discussion: In this experiment, we successfully demonstrated remote control of an LED and a servo motor using a Raspberry Pi and the ThingSpeak platform. By observing the attached ThingSpeak graphs, we can see real-time changes in the data corresponding to the control of the LED and servo motor. The first chart (Field 1) represents the LED control, where the data indicates binary states (0 for off and 1 for on). The LED was turned on and off multiple times, as reflected in the fluctuations between 0 and 1 on the graph.

The second chart (Field 2) illustrates the servo motor's position changes, which range from 0 to around 300 degrees. The sharp increases and decreases in the chart demonstrate successful remote adjustment of the servo's position based on the data sent from ThingSpeak. These real-time updates confirm that the Raspberry Pi is accurately receiving and executing the commands from the cloud platform.

The fluctuations in both charts highlight how the system can handle varying inputs from ThingSpeak and reflect the changes both visually (in the graphs) and physically (in the hardware setup).

Conclusion: This experiment successfully demonstrated the integration of IoT by remotely controlling hardware (an LED and a servo motor) using a Raspberry Pi and ThingSpeak. The real-time graphs on ThingSpeak validate the system’s capability to interact with physical devices over the internet, showcasing the potential of IoT in practical applications. The data was successfully transmitted and updated in ThingSpeak, with accurate reflections of the LED's state and servo motor's position on the attached graphs. This proves the effectiveness of using cloud platforms like ThingSpeak for remote monitoring and control in IoT systems.

Roll Number: 21BIT191